Projet Fil Rouge 2

RAPPORT ET CODE SOURCE

Groupe N° : 3

Membres du groupe :

- BACHAA Hajar
- BEN LTAIF Yasmine
- DEVAUD Antoine
- GRELET Thomas
- YAHYAOUI Nidal

Tuteur :

- VANDERSTRAETEN Julien

Année universitaire 2025-2026

Sommaire

1. Introduction et Mise en Contexte....3
1.1. Présentation du PFR-Semestre 2.... 3
1.2. Objectifs et Défis du Semestre 2....3
1.3. Environnement de travail.... 3
2. Architecture du système.... 4
2.1. Architecture matérielle.... 4
2.2. Architecture logicielle.... 4
2.3. Montage matériel et positionnement.... 5
3. Mode de pilotage.... 7
3.1. Pilotage par commande clavier.... 7
3.2. Pilotage par commande vocale.... 7
3.3. Pilotage par Joystick (Interface Web).... 7
3.4. Mode automatique....7
4.Fonctionnalités détaillées.... 8
4.1. Cartographie LIDAR....8
4.2. Détection de balles.... 8
4.3. Interface web....8
4.3. Communication Série Arduino.... 9
5. Validation et Résultats.... 9
5.1. Scénarios de tests effectués....9
5.2. Analyse des résultats.... 10
5.3. Limites rencontrées.... 10
6. Gestion de Projet et Organisation....10
6.1. Répartition des tâches au sein du groupe....10
6.2. Outils de collaboration....11
6.3. Planning....11
7. Manuel d'utilisation....11
7.1. Prérequis.... 11
7.2. Lancement du système.... 11
8. Bilans Individuels....13
8.1. Bilan de BACHAA Hajar.... 13
8.2. Bilan de BEN LTAIEF Yasmine.... 14
8.3. Bilan de DEVAUD Antoine....15
8.4. Bilan de GRELET Thomas....17
8.5. Bilan de YAHYAOUI Nidal....18
9. Conclusion....19
9.1. Bilan général du projet....19
9.2. Perspectives d'évolution....20

1. Introduction et Mise en Contexte

1.1. Présentation du PFR-Semestre 2

Le Projet Fil Rouge du semestre 2 constitue la phase de concrétisation physique du projet initié au semestre 1. Alors que le premier semestre était centré sur le développement logiciel en simulation (langage C, Python, traitement d'image), le semestre 2 a pour objectif le déploiement de l'ensemble du système sur un robot physique réel.

Le robot doit pouvoir être piloté de plusieurs manières : par commande clavier, par commande vocale, par joystick via une interface web, et de manière autonome par détection de balles colorées. Il doit également cartographier son environnement grâce à un capteur LiDAR.

1.2. Objectifs et Défis du Semestre 2

Les principaux objectifs de ce semestre étaient :

- Intégrer le code C développé au semestre 1 sur la Raspberry Pi
- Mettre en place la communication entre la Raspberry Pi et l'Arduino Mega via port série USB
- Développer une interface web de pilotage accessible depuis un navigateur
- Implémenter la commande vocale avec reconnaissance vocale
- Réaliser la cartographie LiDAR en temps réel
- Développer la détection et le suivi de balles colorées

1.3. Environnement de travail

Pour mener à bien ce projet, notre groupe a utilisé les outils et technologies suivants:

- Système embarqué : Raspberry Pi 3 sous Raspberry Pi OS (Linux)
- Microcontrôleur : Arduino Mega 2560
- Accès distant : SSH via réseau Wi-Fi
- Langage C : programme principal main.c compilé via Makefile
- Python : scripts Flask, pilote_robot.py, scan_lidar.py, server_rpi.py
- Interface web : HTML/CSS/JavaScript avec Flask comme serveur
- Gestion de version : GitHub pour le travail collaboratif

2. Architecture du système

2.1. Architecture matérielle

Notre système repose sur deux unités principales qui communiquent entre elles. Cette séparation permet de distinguer clairement les traitements intelligents des traitements temps réel.

2.1.1. La Raspberry Pi — Cerveau du système

La Raspberry Pi joue le rôle de centre de contrôle principal. Elle héberge l'ensemble des programmes principaux et gère :

- Le programme principal en C (main.c) avec son menu à 6 options
- Le serveur Flask (server_rpi.py) pour l'interface web
- Le script de pilotage (pilote_robot.py) qui traduit les actions en octets Arduino
- Le script de cartographie LiDAR (scan_lidar.py + display_map.py)
- La caméra Pi pour la détection de balles colorées
- La communication SSH avec le PC de contrôle

2.1.2. L'Arduino Mega — Muscles du système

L'Arduino Mega gère la partie matérielle en temps réel. Elle reçoit des octets de commande de la Raspberry Pi via le port série USB et contrôle :

- Les 4 moteurs via le shield moteur (qui fournit la puissance nécessaire)
- Les 2 capteurs ultrason (avant et arrière) pour la détection d'obstacles
- Le module Bluetooth HC-05 pour le contrôle sans fil
- Le haut-parleur utilisé comme klaxon (signal sonore de communication)

2.2. Architecture logicielle

L'architecture logicielle suit une approche en couches, où chaque niveau communique avec le suivant via des fichiers texte partagés ou des connexions réseau.

2.2.1. Couche Interface (PC / Navigateur)

L'utilisateur interagit avec le robot via une interface web (interface_robot.html) ouverte dans un navigateur sur le PC. Cette interface propose :

- Un joystick numérique pour le contrôle temps réel (envoi direct d'octets à l'Arduino), des boutons pour modifier la vitesse du robot et un bouton pour “klaxonner”
- Un champ de texte pour envoyer des commandes textuelles
- Un bouton microphone pour envoyer des commandes vocales
- Un flux caméra en direct ou le résultat des images prises lors de la détection de balles
- Un bouton pour activer/désactiver le mode autonome dit “aspirateur”
- Une image résultat de la cartographie LiDAR à un moment précis
- Un journal des commandes envoyées

2.2.2. Couche Serveur (Raspberry Pi — Flask)

Le serveur Flask (server_rpi.py) fait le lien entre l'interface web et le code de pilotage. Il expose les routes suivantes :

- /joystick : envoie un octet directement à l'Arduino pour le contrôle temps réel
- /commande : interprète une commande naturelle et écrit dans action.txt
- /audio : reçoit un enregistrement audio, effectue la reconnaissance vocale, et écrit dans action.txt
- /signal : exécute pilote_robot.py qui lit action.txt et pilote l'Arduino
- /video : diffuse le flux caméra en temps réel
- /status : retourne l'état du serveur et de l'Arduino
- /etat : retourne l'état de la détection de balle
- /carte : retourne l'image de la carte LiDAR
- /lidar_log : stream des données LiDAR en temps réel
- /reset : arrêt forcé du robot et vidage des fichiers
- /status : retourne l'état du serveur et de l'Arduino

2.2.3. Couche Traitement C (Raspberry Pi)

Le programme principal en C (main.c) propose un menu interactif à 6 options :

- Option 1 — Écrire une commande (clavier) : saisie_clavier() + traiter_commande()
- Option 2 — Parler (commande vocale) : lance assistant_vocal.py + traiter_commande()
- Option 3 — Lancer la simulation(turtle) : lance simulation.py
- Option 4 — Lancer le robot : lance pilote_robot.py
- Option 5 — Lancer la cartographie LiDAR : lance scan_lidar.py | display_map.py
- Option 6 — Arrêter le programme

Un thread de surveillance (surveiller_web) tourne en arrière-plan et détecte les modifications du fichier data/commande.txt écrites par l'interface web, puis appelle automatiquement traiter_commande().

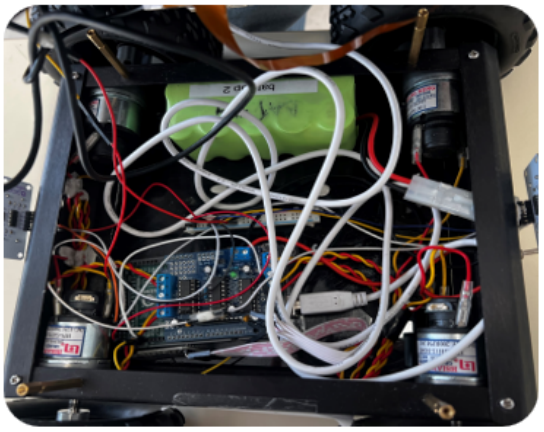
2.3. Montage matériel et positionnement

Le positionnement de chaque composant a été réfléchi pour optimiser les performances du robot.

2.3.1. Intérieur du châssis (niveau bas)

À l'intérieur du châssis, on trouve :

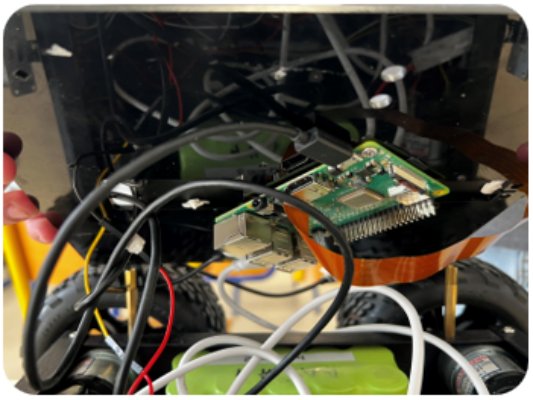
- Les 4 moteurs placés aux quatre coins du châssis
- Le shield moteur connecté à l'Arduino (fournit la puissance nécessaire aux moteurs)
- L'Arduino Mega avec sa carte Bluetooth
- La batterie dédiée aux moteurs placée sur le côté



2.3.2. Plateforme supérieure (niveau haut)

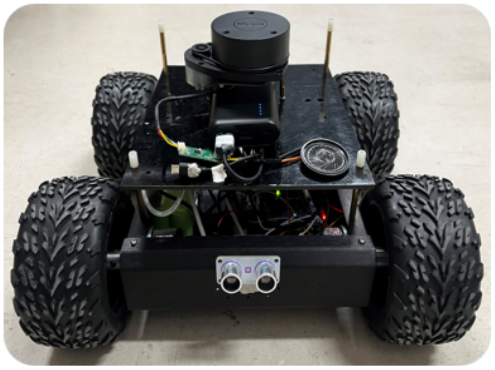
La plateforme supérieure comporte deux faces :

- Face intérieure : la Raspberry Pi fixée sous la plateforme



- Face extérieure (visible) : la batterie externe alimentant la Pi, le haut-parleur, et le LiDAR

Le LiDAR est placé tout en haut du robot pour avoir une vue dégagée à 360°, sans être obstrué par le châssis, ce qui lui permet de cartographier correctement toute la salle.



2.3.3. Façade avant et arrière

Les deux capteurs ultrason sont placés à l'avant et à l'arrière du robot. Ce choix est justifié par le fait que le robot se déplace principalement en avançant et en reculant . La caméra est positionnée à l'avant en hauteur pour avoir un meilleur angle de vision sur les objets au sol.

3. Mode de pilotage

3.1. Pilotage par commande clavier

Le mode clavier permet à l'utilisateur de saisir des commandes textuelles directement dans le terminal ou dans l'interface web. Le module commande_vocale.c analyse la phrase saisie mot par mot en se référant aux fichiers JSON de configuration (fr.json, en.json, es.json) pour reconnaître les actions.

Les commandes supportées sont :

- advance N : avancer de N mètres
- retreat N : reculer de N mètres
- turn left/right N : tourner à gauche ou à droite de N degrés
- find ball couleur : chercher une balle de la couleur spécifiée
- stop : arrêt d'urgence

Le système supporte les commandes complexes multi-actions séparées par des mots de liaison (et, puis, ensuite).

3.2. Pilotage par commande vocale

Le mode vocal utilise la bibliothèque Python SpeechRecognition pour capturer la voix via le microphone du PC, et l'API Google STT pour la transcription en texte. Le flux est le suivant :

- L'utilisateur parle dans le microphone du PC
- Le script Python enregistre l'audio et l'envoie à Google Speech-to-Text
- Le texte reconnu est envoyé à la Raspberry Pi via SSH ou Flask
- Le module commande_vocale.c interprète le texte et génère les actions dans action.txt
- pilote_robot.py exécute les actions sur l'Arduino

La reconnaissance vocale fonctionne en français, anglais et espagnol grâce aux fichiers fr.json, en.json et es.json.

3.3. Pilotage par Joystick (Interface Web)

Le joystick numérique de l'interface web permet un contrôle temps réel du robot. Lorsque l'utilisateur maintient un bouton appuyé (haut, bas, gauche, droite), le script JavaScript envoie des requêtes POST répétées à la route /joystick du serveur Flask. Flask transmet directement l'octet de commande à l'Arduino via le port série .

Le contrôle par joystick est le seul mode où Flask communique directement avec l'Arduino, sans passer par action.txt. Cela garantit une réponse en temps réel.

3.4. Mode automatique

Le mode automatique permet au robot de naviguer de manière autonome dans son environnement sans intervention humaine. Dans ce mode, le robot avance en continu et réagit automatiquement aux obstacles détectés par les capteurs ultrason.

La chaîne de traitement est la suivante :

- L'Arduino lit en permanence les valeurs des 2 **capteurs ultrasons** (avant et arrière)
- Si un **obstacle** est détecté devant le robot à une **distance inférieure** au seuil défini, le **robot s'arrête** automatiquement
- Il effectue ensuite une rotation à gauche jusqu'à avoir une distance supérieur
- Il **reprend** son **avance** dans la nouvelle direction
- Ce processus se répète en boucle tant que le mode automatique est actif

Ce mode est activé depuis l'interface web via le bouton "MODE ASPIRATEUR" qui envoie l'octet '10' à l'Arduino. Il est désactivé en renvoyant l'octet '3' (stop) en appuyant sur son bouton associé.

4.Fonctionnalités détaillées

4.1. Cartographie LIDAR

La cartographie est réalisée par le capteur **RPLidar** connecté à la Raspberry Pi. Le script scan_lidar.py lit les données du capteur (angle, distance) en temps réel et les transmet via un pipe Unix au script display_map.py qui génère une représentation graphique 2D de l'environnement.

Le LiDAR effectue un scan à 360°, permettant de détecter les obstacles et les murs tout autour du robot.

4.2. Détection de balles

La détection utilise la bibliothèque **OpenCV** sur la Raspberry Pi. Le processus de détection suit ces étapes dans pilote_robot.py :

- Acquisition d'une image (prise par la caméra)
- Conversion de l'image de BGR vers **HSV**
- Application d'un **masque** de couleur selon les plages HSV définies par couleur
- Flou médian pour **réduire le bruit**
- **Détection** des contours et calcul de la **circularité**
- Sélection du meilleur contour circulaire (circularité >= 0.8, aire > 500)
- **Calcul du centre** (cx, cy) et du **rayon** de la balle

Les couleurs supportées sont : **rouge, bleu, vert** et **jaune**, chacune définie par ses plages HSV spécifiques.

4.3. Interface web

L'interface web (interface_robot.html) est une application **HTML/CSS/JavaScript** qui s'ouvre dans un navigateur Chrome sur le PC. Elle **communique** avec le serveur Flask de la Raspberry Pi via **des requêtes HTTP**.

Les fonctionnalités de l'interface sont :

- **Indicateurs d'état** (Serveur, Arduino, Caméra) mis à jour toutes les 2 secondes
- **Flux caméra** en direct en format MJPEG
- **Joystick numérique** avec support clavier (flèches + espace)
- **Onglet Texte** : saisie de commandes naturelles avec envoi et lancement
- **Onglet Vocal** : enregistrement audio via MediaRecorder et envoi à Flask
- **Journal des commandes** avec horodatage

4.3. Communication Série Arduino

La communication entre la Raspberry Pi et l'Arduino Mega se fait via le port USB série (/dev/ttyACM0) à 9600 bauds. Le script pilote_robot.py lit le fichier action.txt et traduit chaque action en octet envoyé à l'Arduino :

- Octet '1' + durée : avancer
- Octet '2 '+ durée : reculer
- Octet '3' : stop
- Octet '4' + durée : tourner à gauche
- Octet '5' + durée : tourner à droite
- Octet '6' : augmenter la vitesse de 10%
- Octet '7' : réduire la vitesse de 10%
- Octet '8' : klaxon
- Octet '10' : pilotage automatique

Les durées sont calculées en fonction des distances et angles demandés, grâce à des formules de calibration établies lors des tests.

5. Validation et Résultats

5.1. Scénarios de tests effectués

- **Test 1 : Pilotage multimodal**
  - *Objectif* : Vérifier que les 4 modes de pilotage (clavier, vocal, joystick, automatique) fonctionnent correctement et que les commandes sont bien transmises à l'Arduino.
  - *Résultat* : Le joystick fonctionne en temps réel. Les commandes texte et vocale sont interprétées et exécutées correctement.Le mode automatique permet au robot de naviguer seul en évitant les obstacles.
- **Test 2 : Détection de balle bleue**
  - *Objectif* : Commander au robot de trouver une balle bleue dans son environnement.
  - *Résultat* : La commande 'trouve la balle bleue' est reconnue. La caméra capture l'image, détecte la balle, calcule l'angle et la distance, et le robot s'en approche automatiquement.
- **Test 3 : Cartographie LiDAR**
  - *Objectif* : Générer une carte 2D de la salle de cours.
  - *Résultat* : Le LiDAR génère une carte en temps réel de l'environnement à 360°. Les obstacles et les murs sont correctement identifiés.
- **Test 4 : Interface Web**

- *Objectif* : Contrôler le robot depuis l'interface web avec flux caméra en direct.
- *Résultat* : L'interface affiche le flux caméra en temps réel. Le joystick, les commandes texte et vocales fonctionnent depuis le navigateur.

5.2. Analyse des résultats

Les tests démontrent que l'intégration matérielle et logicielle est fonctionnelle. La communication entre les trois couches (PC, Raspberry Pi, Arduino) est stable. La détection de balles fonctionne dans des conditions de luminosité standard. Le flux caméra est fluide à 20 images par seconde.

5.3. Limites rencontrées

- Capteurs ultrason peu fiables : bruit parasite sur certaines surfaces, génération de fausses alarmes
- Luminosité variable : les seuils HSV de détection de couleur doivent être ajustés selon l'éclairage
- Sol différent : la calibration des durées de déplacement varie selon le type de sol
- Batterie défaillante en cours de tests : interruptions lors des démonstrations
- Synchronisation SSH : latence parfois élevée pour les commandes complexes
- Arduino initiale défectueuse

6. Gestion de Projet et Organisation

6.1. Répartition des tâches au sein du groupe

Le groupe a été divisé en deux équipes spécialisées après une phase initiale de test du matériel :

Équipe Raspberry Pi

- Développement du serveur Flask et de l'interface web
- Intégration de la caméra Pi et du LiDAR
- Commande vocale et reconnaissance Google STT
- Script pilote_robot.py

Équipe Arduino

- Programmation de l'Arduino Mega
- Gestion des 4 moteurs via shield moteur
- Intégration des capteurs ultrason
- Module Bluetooth et haut-parleur

6.2. Outils de collaboration

- **GitHub** : Notre outil principal pour le versioning. Il nous a permis de partager le code source, de suivre les modifications et de garantir que tout le monde travaille sur la dernière version des fichiers
- **SSH** : accès distant à la Raspberry Pi depuis les PC personnels
- **Instagram** : Utilisé comme canal de communication rapide. Ce choix nous a permis une grande réactivité pour les prises de décision immédiates, le partage de captures d'écran de bugs et la coordination des sessions de travail.

6.3. Planning

Le semestre 2 a été organisé en plusieurs phases :

- Phase 1 : Test du matériel et identification des pannes
- Phase 2 : Développement parallèle des équipes Pi et Arduino
- Phase 3 : Intégration et tests de communication Pi-Arduino
- Phase 4 : Développement de l'interface web et de la commande vocale
- Phase 5 : Tests finaux, calibration et préparation de la soutenance

7. Manuel d'utilisation

7.1. Prérequis

- PC avec navigateur Chrome et connexion Wi-Fi
- Raspberry Pi allumée et connectée au même réseau Wi-Fi
- Arduino Mega branchée sur la Raspberry Pi via USB
- Batterie du robot chargée

7.2. Lancement du système

Étape 1 — Connexion à la Raspberry Pi via VNC

Ouvrir le client VNC sur le PC et se connecter à l'adresse 172.20.10.8 avec l'utilisateur pfr3. Cela ouvre le bureau graphique de la Raspberry Pi à distance.

Étape 2 — Lancement du serveur Flask (Terminal 1)

Dans le bureau VNC, ouvrir un premier terminal, se placer dans le dossier du projet et lancer le serveur Flask :

```
cd ~/code/PFR_groupe3
python server_rpi.py
```

Attendre l'affichage de Running on http://172.20.10.8:5000 avant de continuer.

Étape 3 — Lancement du programme principal (Terminal 2)

Ouvrir un deuxième terminal dans VNC, se placer dans le dossier du projet et compiler puis lancer le programme C :

```
cd ~/code/PFR_groupe3
make re
```

Étape 4 — Ouverture de l'interface web

Sur le PC, ouvrir Google Chrome, ouvrir le fichier interface_robot.html, entrer l'adresse 172.20.10.8:5000 dans le champ IP et cliquer sur CONNECTER.

8. Bilans Individuels

8.1. Bilan de BACHAA Hajar

La réalisation de ce projet m'a permis de découvrir concrètement les différentes étapes de conception d'un robot intelligent, depuis le montage matériel jusqu'au développement logiciel.

Ma contribution personnelle a porté sur le développement du script assistant_vocal.py permettant la réception des commandes vocales sur la Raspberry Pi, ainsi que test_voix.py qui capture la voix depuis le microphone du PC et l'envoie via SSH. J'ai également contribué au développement de pilote_robot.py, participé au montage physique du robot et aux phases de tests et d'intégration.

Ce travail m'a permis d'approfondir mes connaissances dans plusieurs domaines, notamment :

- les systèmes embarqués,
- la robotique,
- la communication entre la Raspberry Pi et l'Arduino,
- le contrôle des moteurs,
- ainsi que l'utilisation des capteurs et du LiDAR.

J'ai également appris à intégrer plusieurs composants matériels dans un même système tout en assurant leur bon fonctionnement et leur communication.

Parmi les principales difficultés rencontrées, je peux citer :

- les problèmes de connexion entre certains composants,
- la gestion de l'alimentation des moteurs,
- ainsi que la configuration et l'intégration des différents modules du robot.

Cependant, ces difficultés m'ont permis de développer ma capacité d'analyse, de recherche et de résolution de problèmes.

Ce projet a aussi renforcé mon organisation ainsi que mes compétences pratiques dans le domaine de la robotique et des systèmes intelligents.

Enfin, cette expérience a été très enrichissante car elle m'a permis d'appliquer les connaissances théoriques acquises durant ma formation dans un projet concret et fonctionnel.

Ce projet m'a également motivée à approfondir davantage mes connaissances dans le domaine de la robotique.

8.2. Bilan de BEN LTAIEF Yasmine

Travail effectué

En début de projet, j'ai configuré les outils d'accès à distance à la Raspberry Pi — PuTTY pour le terminal SSH et VNC Viewer pour l'interface graphique. J'ai ensuite participé aux tests matériels de la Raspberry Pi et de la caméra, puis au montage physique complet du robot et à la définition de son architecture avec le reste du groupe.

Pour la partie logicielle, j'ai choisi de me concentrer sur la commande vocale pour cette deuxième partie. L'objectif était de permettre au robot de recevoir et d'exécuter des commandes vocales en conditions réelles.

Face aux contraintes rencontrées, j'ai été amenée à développer une interface web comme solution de contrôle, intégrant la commande vocale, le flux caméra en temps réel, le joystick virtuel, les commandes texte, la cartographie LiDAR et la gestion des différents modes du robot. Cela a également nécessité la mise en place d'un serveur Flask sur la Raspberry Pi pour assurer la communication avec le programme C existant.

Le cœur du travail a consisté à mettre en place une communication entre l'interface web et le programme C existant. Pour cela, j'ai développé un serveur Flask en Python tournant sur la Raspberry Pi, qui joue le rôle de pont : il reçoit les commandes envoyées depuis l'interface, les écrit dans un fichier partagé lu par le programme C, qui les traite et génère les actions à envoyer à l'Arduino. J'ai également développé l'interface web en HTML, CSS et JavaScript, qui centralise tous les moyens de contrôle du robot : commande vocale via le microphone du navigateur, commande texte, joystick virtuel pour le contrôle direct de l'Arduino, flux caméra en temps réel, cartographie LiDAR avec affichage progressif, et gestion des modes du robot comme le mode automatique et l'arrêt forcé.

Difficultés rencontrées

La principale difficulté a porté sur la commande vocale. Le code tourne sur la Raspberry Pi qui ne dispose pas de microphone, et intégrer un micro directement sur le robot n'était pas envisageable car il est mobile. J'ai exploré plusieurs alternatives, notamment des applications permettant de transformer un téléphone en microphone, mais elles n'étaient pas compatibles avec la Raspberry Pi ou ne fonctionnaient pas correctement, faire tourner la reconnaissance vocale sur le PC et envoyer les données à la RPi nous a obligeait à jongler en permanence entre deux terminaux, ce qui n'était pas pratique en conditions réelles. C'est de cette contrainte qu'est née l'idée de développer une interface web, qui permet d'utiliser le microphone du navigateur sur n'importe quel appareil et d'envoyer les commandes au robot via le réseau Wi-Fi, sans matériel supplémentaire et sans contrainte de mobilité.

Compétences acquises

Ce projet m'a permis d'approfondir mes connaissances sur les systèmes embarqués et leur architecture, notamment la communication entre la Raspberry Pi et l'Arduino, la gestion des périphériques matériels et les contraintes liées au déploiement d'un logiciel sur un système embarqué mobile. J'ai également mieux compris les interactions entre les couches matérielles et logicielles d'un système robotique complet. Sur le plan du travail en équipe, collaborer avec cinq personnes sur un projet intégrant autant de composants matériels et logiciels m'a appris l'importance d'une bonne coordination, d'une répartition claire des rôles et d'une communication régulière pour assurer une intégration cohérente de l'ensemble du système.

8.3. Bilan de DEVAUD Antoine

Travail effectué au cours du projet

Travail initial

Au cours de cette deuxième partie du projet, mon rôle a évolué au fur et à mesure de son avancée. Initialement, je devais m'occuper de tester la carte arduino, les capteurs à ultrason, le shield, le module bluetooth et les 4 moteurs du robot, cela passe alors par la création d'un code arduino qui permet de conduire le robot avec l'aide du téléphone en utilisant l'arrêt quand le robot s'approche du mur.

Ce code a été modifié tout au long de l'avancée du projet, car elle nécessitait des ajouts réguliers au fur et à mesure de l'intégration des autres fonctionnalités (pilotage automatique, klaxon, communication avec raspberry, interface de pilotage) afin de les implémenter dans les commandes du robot.De plus j'ai pris en charge les premiers branchements du robot en connectant les composants principaux entre eux

Travail de conception et d'intégration

Après cette mise en place initiale, j'ai commencé à travailler avec Thomas sur la création de la fonctionnalité de la détection de balle pour que le robot puisse pouvoir trouver et s'approcher de la balle de la couleur demandée. J'ai alors travaillé sur l'utilisation d'OpenCV afin d'identifier une balle de couleur dans une photo capturée par la caméra de la Raspberry Pi. Cela passe par une conversion de l'espace colorimétrique BGR vers HSV, l'application d'un masque de couleur, un nettoyage morphologique du masque, puis la détection de contours pour localiser la balle via une formule sur la circularité de la forme. À partir du contour le plus grand(si circularité >0,8) on calcule le centroïde précis via les moments d'image, ce qui permet d'obtenir les coordonnées du centre de la balle dans le repère image.

Une fois les coordonnées ce centre obtenu, j'ai mis en place le calcul de l'angle horizontal entre l'axe de la caméra et la position de la balle, en utilisant la longueur focale exprimée en pixels, elle-même dérivée du champ de vision horizontal de la caméra. Cet angle est ensuite converti en durée de rotation pour l'Arduino (formule

trouvée avec des expérimentations), permettant au robot de s'orienter précisément vers la balle en une seule prise de photo, sans boucle de recadrage coûteuse en temps. De plus, on a fait de même pour calculer la distance entre le robot avec la balle en prenant comme repère le rayon de la balle. Une fois ses variables connues, il n'y a plus qu'à effectuer l'action nécessaire. Au final l'ajout que le robot fasse un tour sur lui-même tant qu'il ne trouve pas de balle permet de nettoyer un grand champ de vision et permet au robot de trouver au maximum ce qu'on lui demande.

Ajustements finaux et tests

Enfin, la phase de tests a nécessité de nombreux ajustements, notamment sur les plages HSV pour chaque couleur selon les conditions d'éclairage, sur la constante de vitesse de rotation du robot pour que la conversion angle-durée soit fiable, et sur les seuils de surface minimale pour éviter les fausses détections. De plus, la gestion de la distance de parcours à faire selon la balle détectée. Ces tests ont impliqué toute l'équipe, chacun testant son module dans des conditions réelles afin d'identifier et corriger les problèmes au maximum en notant alors que l'environnement complexifie grandement la tâche.

Difficultés rencontrées et compétences acquises

Programmation et expérimentation

Ce semestre m'a permis de progresser significativement sur plusieurs aspects techniques. J'ai découvert et pris en main la bibliothèque OpenCV en Python, en partant des bases de la manipulation d'images jusqu'à des notions plus avancées comme les moments d'image, les opérations morphologiques et la projection géométrique pour le calcul d'angles. Travailler sur un cas concret, avec une caméra réelle et des conditions d'éclairage variables, m'a appris à rendre un algorithme de vision robuste plutôt que simplement fonctionnel en conditions idéales.

J'ai également approfondi ma maîtrise de la communication série entre deux systèmes embarqués, en gérant les contraintes de timing et de synchronisation entre la Raspberry Pi et l'Arduino, ce qui demande une bonne compréhension des deux côtés de la chaîne.

C'est pourquoi j'ai bien compris la différence avec la réalité , avec beaucoup d'inconnus qui entrent en compte pour différentes choses : l'environnement du robot est primordial à prendre en compte, sol , obstacle, luminosité etc…

Gestion d'équipe

Par rapport au premier semestre, la coordination avec mon équipe a été plus fluide, grâce aux habitudes de travail et aux outils mis en place. L'utilisation de GitHub est devenue naturelle, et la répartition des rôles plus claire et plus efficace. Cependant, l'intégration de modules développés en parallèle reste un défi complexe: faire fonctionner ensemble des composants codés séparément demande une communication constante et des conventions partagées dès le départ, ce que nous avons mieux anticipé cette fois-ci. Cela nous a permis alors d'être plus ou moins dans les temps tout le long du projet avec des mises au point et rendez-vous avec le client récurrents.

8.4. Bilan de GRELET Thomas

Travail effectué

Ce semestre a été découpé en plusieurs phases. La première a été pour moi de tester la carte Arduino Mega grâce à l'IDE Arduino. Je me suis également occupé de tester le capteur LiDAR et à avoir une première visualisation grâce au logiciel Slamtec. En parallèle j'ai formaté la carte SD pour y installer l'OS Raspberry adapté, car je l'avais déjà fait dans d'autres projets donc j'avais déjà les logiciels installés pour réaliser cette tâche. Suite à diverses raisons, cette carte a été reformaté plusieurs fois par les différents membres de l'équipe.

Au milieu du projet j'ai pu commencé à m'intéresser à la modélisation 3D de supports pour les capteurs ultrasons. En effet, l'année dernière j'avais réalisé une formation au FabLab pour pouvoir utiliser les imprimantes 3D donc j'ai pris l'initiative. Finalement, j'ai abandonné cette tâche pour la donner à Nidal.

Ensuite l'objectif a été de relier la Rapsberry à l'Arduino. J'ai donc participé à concevoir un script python qui fait le lien entre les deux (pilote_robot.py). Une fois ce fichier créé, nous étions capables d'envoyer des commandes depuis notre code. Enfin, chaque membre du groupe a pu faire évoluer ce fichier pour y intégrer chaque partie.

Vient ensuite une phase d'expérimentations. L'objectif était de faire avancer le robot d'une certaine distance. J'ai activement participé pour trouver des formules, et les intégrer au code. De même pour la rotation d'un certain angle.

La phase suivante a consisté à développer la fonctionnalité de détection de balles (principalement dans le fichier pilote_robot.py donc), en collaboration avec Antoine. C'est pour moi la suite directe de ce que j'ai fait au premier semestre. L'algorithme développé a été expliqué plus tôt dans le rapport par Antoine.

Enfin, la dernière phase a concerné les tests et ajustements finaux, pour ainsi préparer la présentation finale et la démonstration. Tous les membres de l'équipe ont été impliqués.

En plus de ça, j'ai entretenu des relations avec le client avec de nombreux envois de compte-rendus.

Difficultés rencontrées et compétences acquises

Ayant uniquement des bagages théoriques, l'utilisation du matériel a parfois été laborieuse, j'ai dû tout apprendre sur le tas. Les phases de tests réels ont été particulièrement importantes pour comprendre les limites du matériel et les différences entre une solution fonctionnelle en simulation et un comportement réellement fiable sur un robot physique.

Le code développé au PFR1 sur le traitement de l'image a été inutile ici, car pas vraiment adapté aux conditions réelles. Un peu frustrant quand on a passé plusieurs dizaines d'heures de développement pour qu'au final ça soit remplacé par quelques lignes d'OpenCV, mais cela m'a surtout permis de comprendre l'importance des bibliothèques existantes et l'intérêt de réutiliser des outils optimisés plutôt que de tout développer soi-même.

J'ai également appris à travailler avec des systèmes embarqués réels, notamment la communication entre la Raspberry Pi et l'Arduino, les contraintes liées au temps réel ainsi que les problèmes de calibration des déplacements et des rotations du robot. Les nombreuses phases d'expérimentation m'ont permis de développer une méthode de travail plus rigoureuse basée sur les tests, les mesures et les ajustements progressifs.

Enfin, même si la communication au sein du groupe et avec le client a été assez facile sur cette partie du projet, j'ai tout de même amélioré mes compétences en travail d'équipe et en gestion de projet.

8.5. Bilan de YAHYAOUI Nidal

Dans le cadre du PFR2, j'ai principalement travaillé sur l'intégration matérielle et logicielle du robot, les tests expérimentaux ainsi que la partie cartographie utilisant le LiDAR.

Au début du projet, j'ai participé à la découverte du hardware et à la prise en main des différents composants du robot. Cela comprenait la Raspberry Pi, l'Arduino, les moteurs, le LiDAR, les capteurs ainsi que les systèmes de communication entre les différents modules. Cette phase était importante afin de comprendre le fonctionnement global du robot et les interactions entre les parties électroniques et logicielles.

J'ai également travaillé sur une partie de conception 3D afin de créer des supports de plaques et des fixations pour certains capteurs du robot. L'objectif était d'améliorer l'intégration physique des composants et d'obtenir une structure plus stable et mieux organisée. Plusieurs modèles ont été réfléchis et testés, mais cette partie a finalement été abandonnée par manque de temps et parce que les priorités du projet se sont orientées vers l'intégration et les tests du robot complet.

Une grande partie de mon travail a ensuite porté sur l'intégration des différents modules du projet. J'ai participé à la mise en communication entre les scripts Python, la Raspberry Pi, l'Arduino et les différents capteurs afin que le robot puisse fonctionner correctement dans son ensemble. Cela incluait les tests de déplacements, la récupération des données des capteurs et l'exécution des commandes envoyées au robot.

J’ai aussi travaillé sur la partie cartographie avec le LiDAR. J’ai développé et testé plusieurs scripts permettant de récupérer les données du capteur, traiter les mesures et afficher une représentation 2D de l’environnement. Cette partie demandait beaucoup de tests afin d’obtenir des données exploitables et une visualisation cohérente.

Cette phase a présenté plusieurs difficultés techniques. À un moment du projet, le LiDAR ne fonctionnait plus correctement et j’ai remarqué que certains câbles étaient sectionnés. J’ai donc dû intervenir directement sur le matériel en ressoudant les câbles au FABLAB afin de remettre le capteur en état de fonctionnement. Après cette réparation, plusieurs problèmes restaient présents, notamment du bruit dans les mesures, des pertes de données, des erreurs de communication et des incohérences dans la cartographie lorsque le robot se déplaçait.

Au début, les cartes générées correspondaient surtout à des nuages de points instantanés sans prise en compte du déplacement du robot. J’ai donc essayé d’améliorer progressivement le système afin d’obtenir une cartographie plus réaliste et plus stable pendant les mouvements.

Enfin, j’ai participé aux nombreux tests expérimentaux réalisés durant le projet. Ces essais ont permis de valider les déplacements du robot, les rotations, la communication entre les différents modules ainsi que le comportement global du système dans des conditions réelles. Ces tests ont aussi permis d’identifier et corriger plusieurs problèmes matériels et logiciels liés à l’intégration du robot.

Ce projet m’a permis de découvrir les difficultés concrètes d’un projet robotique complet, notamment l’intégration entre hardware et software, la gestion des problèmes matériels, les contraintes liées aux capteurs ainsi que l’importance des phases de tests dans un système embarqué.

9. Conclusion

9.1. Bilan général du projet

Le semestre 2 du PFR a permis de concrétiser l'ensemble du travail réalisé en simulation au semestre 1. Le robot physique est désormais capable d'être piloté par 4 modes différents (clavier, vocal, joystick, automatique), de cartographier son environnement grâce au LiDAR, et de détecter des balles colorées de manière autonome par vision artificielle.

L'intégration matérielle et logicielle entre la Raspberry Pi, l'Arduino Mega et l'interface web constitue la réalisation principale de ce semestre. La séparation claire entre couche intelligente (Pi) et couche temps réel (Arduino) s'est révélée être une architecture robuste et évolutive.

9.2. Perspectives d'évolution

Plusieurs améliorations pourraient être apportées afin de rendre le robot plus autonome, plus précis et plus robuste.

Tout d'abord, la partie cartographie pourrait être améliorée avec l'intégration d'un véritable système de SLAM (Simultaneous Localization And Mapping). Actuellement, la carte générée repose principalement sur les données du LiDAR et reste limitée lorsque le robot se déplace. L'ajout d'une meilleure estimation de la position du robot grâce à l'odométrie ou à des encodeurs permettrait d'obtenir une cartographie plus stable et plus précise.

La navigation autonome pourrait également être davantage développée. Pour le moment, le robot peut effectuer des déplacements simples et éviter certains obstacles, mais il serait possible d'ajouter des algorithmes plus avancés de planification de trajectoire afin qu'il puisse se déplacer seul vers un objectif précis.

Concernant la vision artificielle, la détection de balles colorées pourrait être améliorée afin de mieux gérer les variations de luminosité, les distances et les environnements plus complexes. L'utilisation de méthodes plus avancées de traitement d'image ou d'intelligence artificielle pourrait rendre la détection plus fiable.

Sur la partie matérielle, plusieurs optimisations restent possibles :

- amélioration du câblage et de la protection des composants électroniques
- création de supports 3D plus robustes et mieux adaptés aux capteurs
- intégration plus compacte et plus propre des différents modules du robot

Enfin, l'interface web pourrait être enrichie avec davantage d'informations en temps réel, comme une visualisation plus avancée de la cartographie, l'affichage des données des capteurs, le suivi de l'état interne du robot ou encore des commandes de contrôle plus complètes. Même si un retour vidéo était déjà présent dans le projet, l'interface pourrait devenir plus interactive et offrir une supervision encore plus détaillée du robot.

Il serait également intéressant de développer un mode utilisateur plus simple et plus intuitif afin qu'une personne ne connaissant pas le fonctionnement technique du robot puisse facilement interagir avec lui. L'objectif serait de proposer une interface plus accessible avec des commandes simplifiées, des indications visuelles claires et davantage d'automatisation pour rendre l'utilisation du robot plus naturelle et plus ergonomique.